

**UNIVERSIDADE FEDERAL DE ITAJUBÁ - UNIFEI**
**CAMPUS ITABIRA**

# RELATÓRIO PARCIAL DO PROJETO MYCASH: QUALIDADE DE SOFTWARE, AUTOMAÇÃO DE TESTES E INFRAESTRUTURA COMO CÓDIGO

**Módulo III: Verificação, Validação e Entrega Contínua**

Frederico Pires de Moraes Gomes - RA: 2023006205
Luiz Otávio de Queiroz Lage Silva - RA: 2023009833
Matheus Lucas Tavares Bueno - RA: 2023013120
Rodrigo Amorim Vieira - RA: 2023010861

**Grupo 9**

Itabira, 2026

Frederico Pires de Moraes Gomes
Luiz Otávio de Queiroz Lage Silva
Matheus Lucas Tavares Bueno
Rodrigo Amorim Vieira

# RELATÓRIO PARCIAL DO PROJETO MYCASH: QUALIDADE DE SOFTWARE, AUTOMAÇÃO DE TESTES E INFRAESTRUTURA COMO CÓDIGO

## Módulo III: Verificação, Validação e Entrega Contínua

Relatório técnico apresentado como requisito parcial para obtenção de aprovação na disciplina Análise e Desenvolvimento de Software IV (ECOi2218), na Universidade Federal de Itajubá – campus Itabira.

Professor:
Prof. Dr. Wendell Fioravante da Silva Diniz

Itabira, 2026

## RESUMO

Este relatório parcial documenta a evolução do projeto MyCash, plataforma de gestão financeira pessoal com integração ao Open Finance brasileiro, durante o módulo dedicado à qualidade de software. Enquanto o módulo anterior entregou a aplicação funcionando em produção, esta etapa concentrou-se em provar que ela funciona, e em tornar reproduzível o ambiente em que ela roda. Foram construídas quatro camadas de verificação automatizada: 222 testes de unidade sobre uma camada de domínio extraída e compartilhada entre os clientes web e móvel, com 100% de cobertura de instruções, ramos, funções e linhas, e escore de mutação de 94,07% verificado com Stryker; 35 testes de integração executados contra um PostgreSQL efêmero provisionado por Testcontainers, incluindo a idempotência da sincronização Open Finance; e 9 testes de ponta a ponta em Playwright, cobrindo dois cenários críticos de negócio com caminho feliz e caminhos alternativos. A infraestrutura foi declarada como código por meio de um *Dockerfile* multiestágio e de um manifesto *Docker Compose* com volume persistente, operável por um único comando. A integração contínua executa três trabalhos paralelos no GitHub Actions a cada envio ao ramo principal. O processo de teste revelou seis defeitos reais em produção, entre eles contas bloqueadas que aceitavam lançamentos, transações do primeiro dia do mês contabilizadas no mês anterior e um gráfico que fabricava os meses anteriores a partir do mês corrente.



**Palavras-chave:** Testes Automatizados. Teste de Mutação. Testcontainers. Playwright. Infraestrutura como Código. Integração Contínua.

# Sumário

**1 INTRODUÇÃO** **4**
1.1 Memorial do desenvolvimento . . . . . 4

**2 DOCUMENTAÇÃO DO ARTEFATO: APLICAÇÃO WEB** **5**
2.1 Regra de negócio projetada . . . . . 5
2.2 Cobertura do artefato: houve plena capacidade de resolver o negócio? . . . 6

**3 ARQUITETURA** **7**
3.1 Visão macro . . . . . 7
3.2 Detalhamento das camadas e suas integrações . . . . . 8
3.3 A arquitetura serviu ao aplicativo móvel? . . . . . 9

**4 NÍVEL DE PROCESSO DOCUMENTADO: REGISTRO DE LANÇAMENTO** **11**
4.1 Etapa 1 — Autenticação . . . . . 11
4.2 Etapa 2 — Caminho alternativo: credencial inválida . . . . . 12
4.3 Etapa 3 — Dashboard consolidado . . . . . 12
4.4 Etapa 4 — Listagem de transações . . . . . 13
4.5 Etapa 5 — Formulário de lançamento . . . . . 14
4.6 Etapa 6 — Persistência . . . . . 15
4.7 Etapa 7 — Consolidação no saldo . . . . . 16
4.8 Demais telas do domínio . . . . . 18
4.9 Discussão . . . . . 20

**5 RELATÓRIO DE TESTES** **21**
5.1 Ferramentas utilizadas . . . . . 21
5.2 Testes de unidade . . . . . 22
5.2.1 Quantidade e distribuição . . . . . 22
5.2.2 Cobertura . . . . . 23
5.2.3 Cenários alternativos . . . . . 24
5.2.4 Teste de mutação . . . . . 25
5.3 Testes de integração . . . . . 26
5.3.1 Estratégia de isolamento de estado . . . . . 26
5.3.2 Escopo coberto . . . . . 27
5.4 Testes de ponta a ponta . . . . . 28
5.4.1 Cenário crítico 1 — Portal de autenticação . . . . . 28
5.4.2 Cenário crítico 2 — Caminho transacional . . . . . 29

5.4.3 Artefatos de diagnóstico . . . . . 30
5.5 Deliberações sobre o que não entra em cada camada . . . . . 30

**6 ESTRATÉGIA DE IMPLEMENTAÇÃO 30**
6.1 Nuvem ou *on-premises* . . . . . 30
6.2 Infraestrutura como código . . . . . 31
6.2.1 Imagem de produção . . . . . 31
6.2.2 Topologia local . . . . . 33
6.2.3 Configuração e segredos . . . . . 33
6.3 Integração contínua com GitHub Actions . . . . . 34
6.4 Entrega contínua . . . . . 35

**7 CONSIDERAÇÕES PARCIAIS 35**
7.1 Próximos passos . . . . . 36

# 1 INTRODUÇÃO

O MyCash é uma plataforma de gestão financeira pessoal cujo objetivo é centralizar, em uma única interface, a visão financeira completa do usuário: contas bancárias, saldos, transações, metas de poupança e lembretes de vencimento. A sincronização automática das contas é realizada por meio do sistema brasileiro de Open Finance, utilizando a plataforma Pluggy como provedor agregador.

A solução é composta por dois artefatos de software que compartilham o mesmo servidor: uma aplicação web construída em Next.js, hospedada na Vercel, e um aplicativo móvel desenvolvido em React Native com Expo, que consome a mesma API REST. O código-fonte está versionado em repositório único, com 65 *commits* registrados entre março e outubro de 2026.

## 1.1 Memorial do desenvolvimento

O projeto atravessou três fases distintas, descritas a seguir.

**Fase 1 — Produto em produção (módulos anteriores).** A aplicação web foi construída sobre Next.js 14 com App Router, PostgreSQL hospedado no Supabase e autenticação delegada ao Supabase Auth. Foram implementados os cinco recursos de domínio com operações completas de criação, leitura, atualização e remoção, além da integração Open Finance. A segurança dos dados foi implementada no próprio banco, por meio de *Row Level Security*. Ao final desta fase, o produto estava em produção, mas a verificação automatizada limitava-se a 19 testes sobre o cliente HTTP e os repositórios.

**Fase 2 — Aplicativo móvel consumindo a API (agosto de 2026).** Atendendo à sugestão registrada na avaliação parcial anterior, o aplicativo móvel foi reconstruído para consumir a API REST própria, em lugar de acessar o Supabase diretamente. Todas as telas do web foram replicadas: *dashboard*, transações, contas, metas, lembretes, notificações e perfil, com tema claro e escuro, gráfico de seis meses e seletor de data nativo. As rotas da API passaram a aceitar autenticação por *Bearer token*, além do *cookie* de sessão já utilizado pelo navegador.

**Fase 3 — Qualidade e infraestrutura (setembro e outubro de 2026).** É a fase documentada neste relatório. A lógica de negócio foi extraída das telas para uma camada de domínio pura, sem dependência de infraestrutura, o que tornou possível testá-la sem rede e sem banco de dados. Sobre essa camada foram construídas quatro suítes de verificação — unidade, mutação, integração e ponta a ponta — e a infraestrutura de execução foi declarada como código.

A Tabela 1 sintetiza a evolução quantitativa da verificação automatizada ao longo do projeto.

Tabela 1: Evolução da verificação automatizada.

| **Camada de teste** | **Módulo anterior** | **Situação atual** |
|---|---|---|
| Unidade (web) | 19 | 162 |
| Unidade (móvel) | 0 | 60 |
| Integração | 0 | 35 |
| Ponta a ponta | 0 | 9 |
| **Total** | **19** | **266** |
| Cobertura do domínio | não medida | 100% |
| Escore de mutação | não medido | 94,07% |
| Trabalhos de CI | 1 | 3 |

Fonte: Autores (2026).

# 2 DOCUMENTAÇÃO DO ARTEFATO: APLICAÇÃO WEB

O artefato escolhido para documentação detalhada é a **aplicação web**, por três razões. Primeiro, é ela que hospeda a API REST consumida também pelo aplicativo móvel, sendo portanto o provedor do contrato. Segundo, concentra a camada de domínio, a persistência e a totalidade da infraestrutura de implantação. Terceiro, a escolha permite responder diretamente à questão proposta no enunciado: se a arquitetura concebida para o site serviu, ou não, ao aplicativo — tema tratado na Seção 3.3.

## 2.1 Regra de negócio projetada

O domínio do MyCash organiza-se em torno de seis regras centrais, apresentadas na Tabela 2. Todas estão implementadas na camada `src/domain`, isolada de qualquer dependência de infraestrutura.

Tabela 2: Regras de negócio e respectiva implementação.

| **Regra** | **Implementação e verificação** |
|---|---|
| Efeito do lançamento no saldo | Entrada soma, saída subtrai, sempre pelo módulo do valor: o sinal provém do tipo, nunca do número gravado. Em `efeitoNoSaldo`. |
| Edição ajusta apenas a diferença | Alterar valor ou tipo de uma transação aplica o delta, não recalcula o saldo inteiro. Em `deltasDaEdicao`. |
| Conta não operável recusa lançamento | Conta de usuário bloqueado ou inativo não aceita movimentação. Em `exigirContaOperavel`. |
| Valor sempre positivo | Valor zero ou negativo é recusado antes de qualquer persistência, com erro de domínio tipado. Em `validarLancamento`. |
| Dinheiro arredondado em centavos | Toda operação de saldo passa por `emCentavos`, evitando acúmulo de resíduo binário de ponto flutuante. |
| Meta não ultrapassa o objetivo | Aporte que excederia o objetivo é limitado, e a meta é marcada como concluída. Em `aportar`. |

Fonte: Autores (2026).

## 2.2 Cobertura do artefato: houve plena capacidade de resolver o negócio?

A resposta é afirmativa para o núcleo do produto e parcial para a integração Open Finance.

**Plenamente atendido.** O ciclo completo de gestão financeira manual está implementado e verificado: cadastro e autenticação de usuário, gestão de contas bancárias, lançamento de transações com categorização, acompanhamento de metas com aportes, lembretes de contas a pagar e a receber, notificações e consolidação no *dashboard*. Os cinco recursos de domínio expõem as quatro operações canônicas, e o isolamento entre usuários é garantido no banco de dados por *Row Level Security*.

**Requisitos não implementados ou implementados parcialmente.** Por honestidade técnica, registram-se as lacunas conhecidas na Tabela 3.

Tabela 3: Requisitos não implementados e respectiva justificativa.

| **Requisito** | **Situação e justificativa** |
|---|---|
| Open Finance validado de ponta a ponta | O código de sincronização está implementado e o *schema* suporta a integração, porém não houve validação contra a conta *sandbox* da Pluggy. O que está verificado é a tradução dos dados e a idempotência da gravação, não a forma da resposta real do provedor. |
| Contratos dirigidos pelo consumidor | Não há contrato formalizado com ferramenta dedicada (Pact). A assimetria de versões entre web e aplicativo está documentada em registro de decisão arquitetural, com tabela de campos obrigatórios e tolerantes, mas a verificação é manual. |
| Notificações por *push* | As notificações existem como registro no banco e são exibidas nas duas interfaces, porém não há envio por *push* ao dispositivo. |
| Open Finance no aplicativo móvel | A conexão de contas via Open Finance só pode ser iniciada pela interface web; o aplicativo exibe as contas sincronizadas, mas não inicia a conexão. |
| Testes de componente na interface | Não há testes de renderização de componentes React. A verificação da interface ocorre nas extremidades: unidade no domínio e ponta a ponta no navegador. |

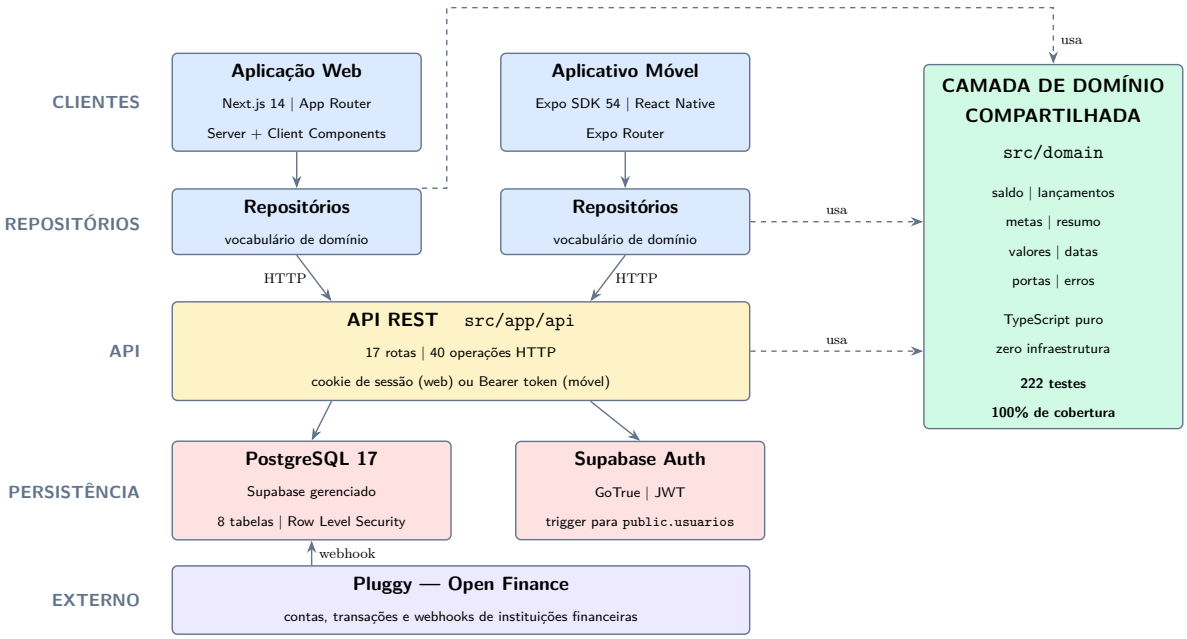
Fonte: Autores (2026).

# 3 ARQUITETURA

## 3.1 Visão macro

A arquitetura segue o princípio da inversão de dependência: o núcleo de regras de negócio não conhece banco de dados, rede ou *framework*; são as camadas externas que dependem dele. A Figura 1 apresenta a visão macro.

Figura 1: Diagrama de arquitetura do MyCash.




Fonte: Autores (2026).

## 3.2 Detalhamento das camadas e suas integrações

**Camada de clientes.** A aplicação web organiza cada tela do *dashboard* como um Server Component, responsável pela busca inicial dos dados no servidor, delegando a interatividade a um Client Component correspondente. O aplicativo móvel, por não dispor de renderização no servidor, busca todos os dados por requisição HTTP a partir de um *hook* próprio, `useRecurso`, que uniformiza estado de carregamento, erro e atualização por gesto de puxar.

**Camada de repositórios.** Encapsula o cliente HTTP e expõe métodos com vocabulário de domínio. O componente invoca `transacoesRepository.criar({idConta, tipo, valor})`, sem conhecimento do *schema* do banco ou do verbo HTTP empregado. É nesta camada que ocorre a tradução entre a notação da interface, em *camelCase*, e a do banco, em *snake_case*.

**Camada de domínio.** É a contribuição arquitetural desta fase. Reúne, em 643 linhas de TypeScript puro, toda a lógica de negócio anteriormente dispersa entre telas e rotas. Não importa Supabase, não executa `fetch` e não conhece React. A dependência é invertida por meio de duas interfaces declaradas em `src/domain/portas.ts`: `ContasPort` e `TransacoesPort`. Em produção, quem as implementa é o adaptador que fala com o PostgREST; nos testes, um repositório em memória. É isso que permite exercitar a regra de negócio sem rede, sem banco e sem contêiner.

Listing 1: Portas do domínio — a dependência apontando para dentro.

```
export interface ContasPort {
  porId(idConta: number): Promise<ContaDoDominio | null>
  ajustarSaldo(idConta: number, delta: number): Promise<void>
}

export interface TransacoesPort {
  porId(idTransacao: number): Promise<TransacaoDoDominio | null>
  inserir(dados: Omit<TransacaoDoDominio, 'id_transacao'>):
     Promise<TransacaoDoDominio>
  atualizar(idTransacao: number, dados:
     Partial<TransacaoDoDominio>): Promise<void>
  remover(idTransacao: number): Promise<void>
}
```

As interfaces são deliberadamente estreitas: declaram o que o domínio usa, e não tudo o que a tabela sabe fazer. Porta larga obrigaria o duplo de teste a implementar métodos que ninguém chama.

**Camada de API.** Dezessete arquivos de rota expõem 40 operações HTTP. A autenticação aceita dois mecanismos: *cookie* de sessão, usado pelo navegador, e cabeçalho `Authorization: Bearer`, adotado quando o aplicativo móvel passou a consumir a mesma API. O formato de resposta é padronizado com *wrapper* `{data}` em caso de sucesso e `{error}` em caso de falha.

**Camada de persistência.** PostgreSQL 17 hospedado no Supabase, com oito tabelas e *Row Level Security* habilitada em todas as tabelas de domínio. O acesso ocorre via PostgREST, por meio da biblioteca `@supabase/ssr`. Não há ORM: as consultas são tipadas pelo arquivo `src/types/database.ts`.

## 3.3 A arquitetura serviu ao aplicativo móvel?

A resposta é afirmativa, mas essa conclusão só se tornou verdadeira após uma correção estrutural realizada nesta fase.

**O problema identificado.** Quando o aplicativo móvel foi construído, as regras de negócio foram reescritas no diretório `mobile/`. Havia, portanto, duas implementações da mesma lógica, mantidas manualmente em paralelo. A extração da camada de domínio revelou que elas haviam divergido em quatro pontos concretos, apresentados na Tabela 4.

Tabela 4: Divergências encontradas entre as implementações web e móvel.

| **Comportamento** | **Divergência observada** |
|---|---|
| Aporte em meta concluída | O web permitia; o aplicativo recusava. |
| Arredondamento | Uma implementação arredondava em centavos; a outra acumulava resíduo de ponto flutuante. |
| Conversão de texto para valor | A entrada `"12abc"` resultava em 12 em uma implementação e em `NaN` na outra. |
| Conta bloqueada | Nenhuma das duas verificava o estado da conta, embora a coluna `status_conta` existisse desde o início do projeto. |

Fonte: Autores (2026).

**A solução adotada.** O diretório `src/domain` foi tornado portável: seus tipos são declarados localmente, em `src/domain/tipos.ts`, em vez de importados de `@/types/database`. Assim, a pasta não depende do *alias* de caminho do Next e pode ser consumida pelo aplicativo móvel. No lado do Expo, bastou acrescentar a pasta aos `watchFolders` do empacotador Metro e declarar o *alias* `@dominio` no `tsconfig.json`. As duas aplicações passaram a importar as mesmas funções.

**Vantagens observadas.** A regra de negócio deixou de existir em duas cópias que divergem com o tempo. Um teste escrito uma única vez passou a proteger os dois clientes. A correção do gráfico do *dashboard* web, descrita na Seção 4.9, consistiu simplesmente em passar a usar a função `serieMensal` que o aplicativo móvel já utilizava.

**Desvantagens e desafios.** O compartilhamento impõe custos reais, registrados a seguir.

- **Árvores de dependência incompatíveis.** O Next exige React 18.3.1 e o Expo SDK 54 exige React 19.1.0. As duas aplicações mantêm `node_modules` separadas, o que obriga a integração contínua a instalar dependências duas vezes, em trabalhos distintos.
- **Fragilidade do empacotador.** O Metro não resolve caminhos fora da raiz do projeto por padrão. Uma primeira configuração, que utilizava `disableHierarchicalLookup`, quebrou a resolução do pacote `expo-asset`; a solução final restringiu-se a declarar `watchFolders`.
- **Acoplamento entre clientes.** Alterar uma função do domínio afeta web e aplicativo ao mesmo tempo. É um acoplamento desejado — é precisamente o objetivo —

mas exige que a suíte de testes seja confiável, pois ela passa a ser a única proteção contra regressão simultânea nos dois produtos.

# 4 NÍVEL DE PROCESSO DOCUMENTADO: REGISTRO DE LANÇAMENTO

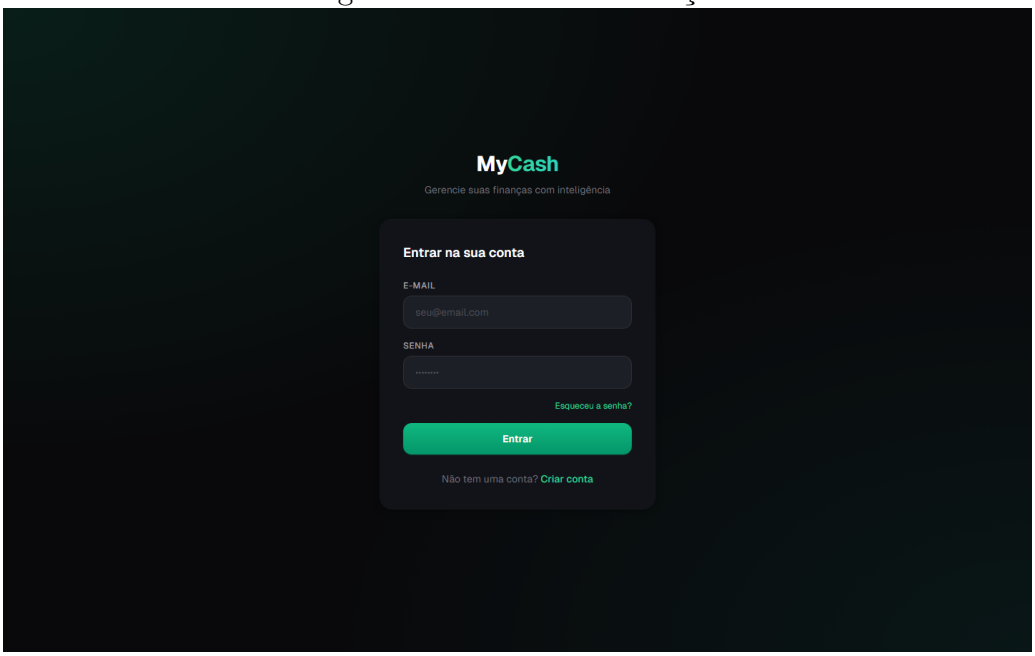
O processo escolhido para documentação detalhada é o **registro de um lançamento financeiro e sua consolidação no saldo**. Trata-se do caminho transacional principal do produto: é por meio dele que o usuário alimenta o sistema, e é dele que derivam todos os números apresentados no *dashboard*.

As capturas apresentadas a seguir foram geradas automaticamente pelo roteiro `e2e/capturar-evidencias.spec.ts`, que percorre o processo com o mesmo Playwright e os mesmos Page Objects utilizados pela suíte de testes. A geração automatizada garante que as evidências não divirjam do comportamento verificado — se a tela muda, o roteiro e os testes quebram juntos.

## 4.1 Etapa 1 — Autenticação

O acesso ao sistema exige sessão válida. Toda rota de dados encontra-se atrás da autenticação, e o isolamento entre usuários é aplicado no banco de dados pelas *policies* de *Row Level Security*.

Figura 2: Tela de autenticação.

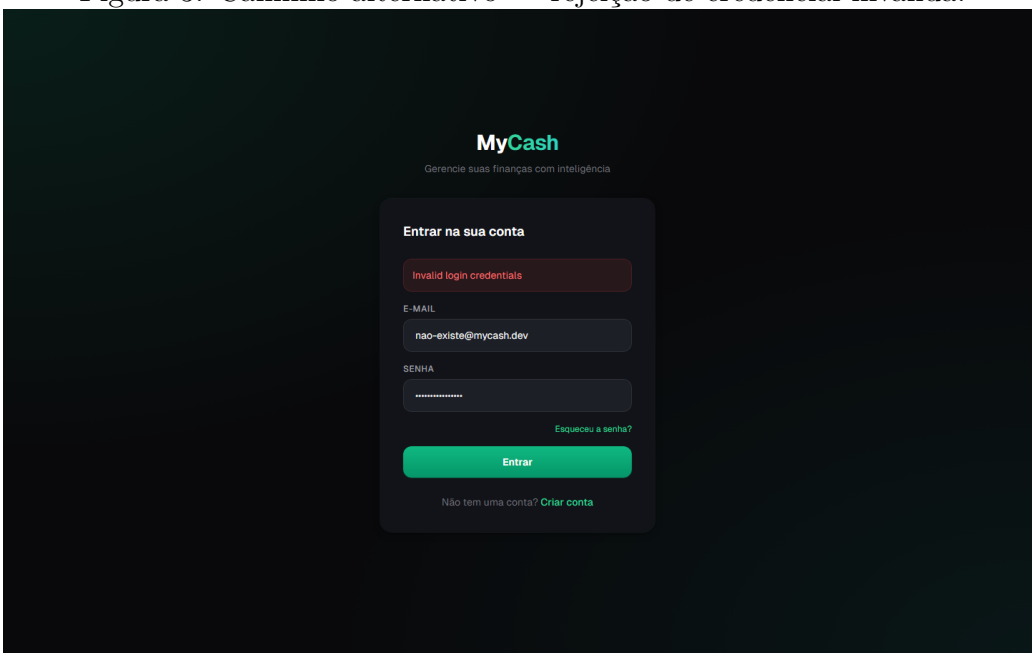



Fonte: Autores (2026).

## 4.2 Etapa 2 — Caminho alternativo: credencial inválida

Submetida uma credencial inexistente, o sistema exibe mensagem de erro e permanece na tela de autenticação. A permanência é tão importante quanto a mensagem: redirecionar ao *dashboard* com sessão inválida constituiria falha de autorização.

Figura 3: Caminho alternativo — rejeição de credencial inválida.

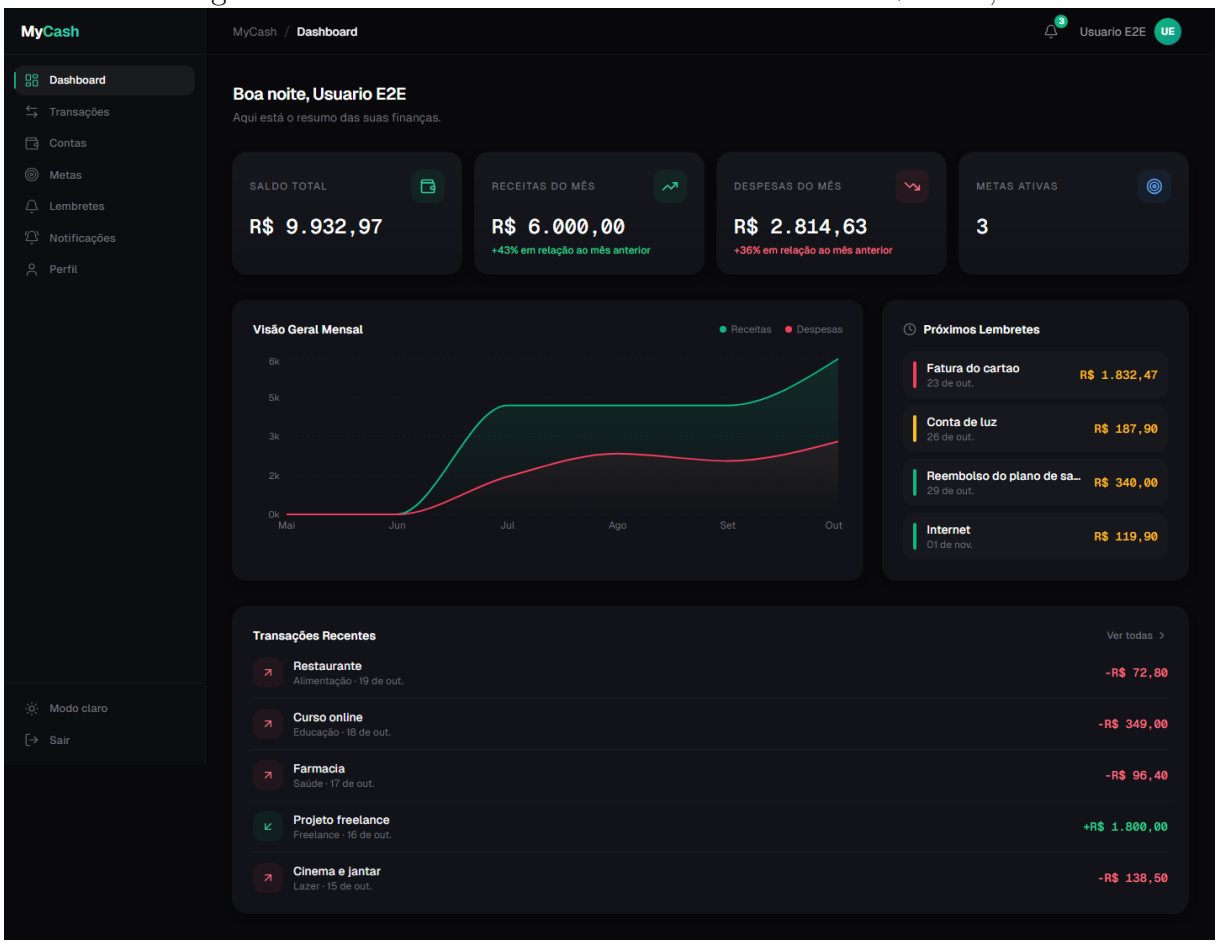



Fonte: Autores (2026).

## 4.3 Etapa 3 — Dashboard consolidado

Autenticado, o usuário acessa o *dashboard*, que apresenta o saldo consolidado de todas as contas, as receitas e despesas do mês com variação percentual real em relação ao mês anterior, a quantidade de metas em andamento, o histórico de seis meses, os próximos lembretes e as transações recentes.

Figura 4: Dashboard com o saldo consolidado em R$ 9.932,97.

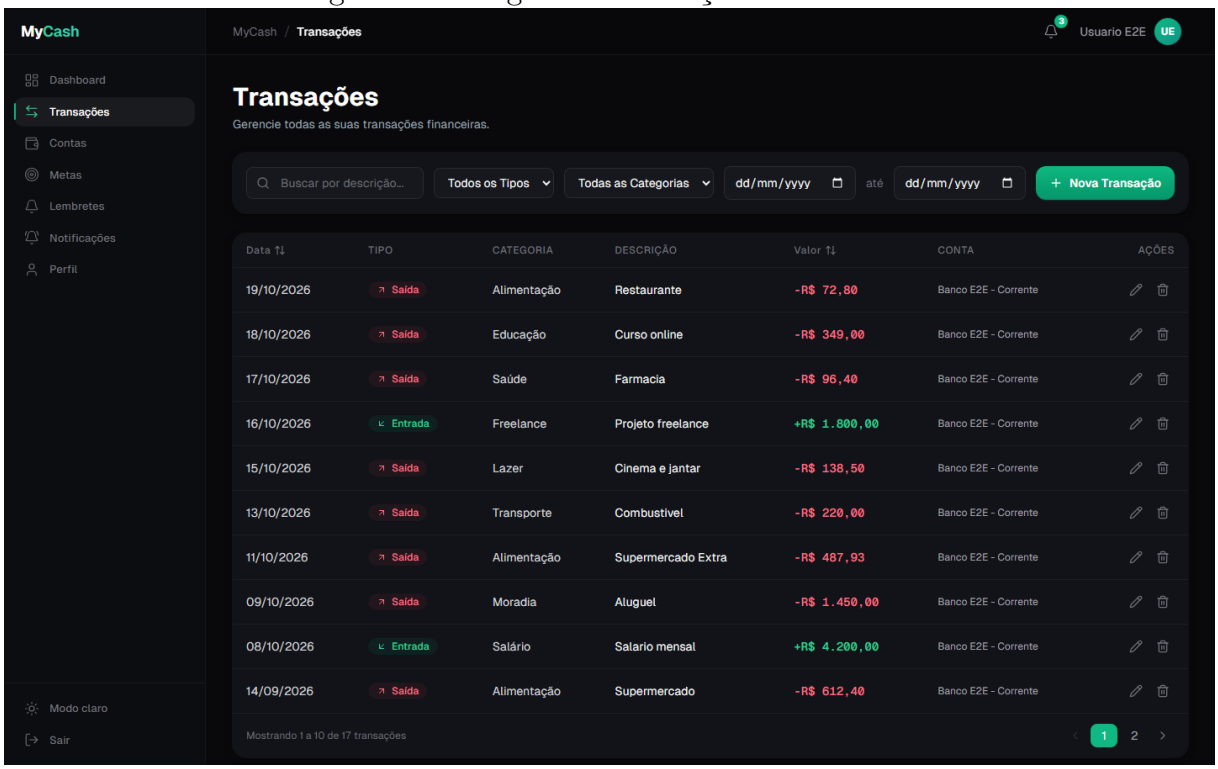



Fonte: Autores (2026).

## 4.4 Etapa 4 — Listagem de transações

A tela de transações oferece filtros por período, tipo, categoria e busca textual. O filtro de período é aplicado no banco de dados, por meio de parâmetros enviados à API; os demais são aplicados localmente, por serem baratos e mudarem a cada tecla digitada.

Figura 5: Listagem de transações com filtros.

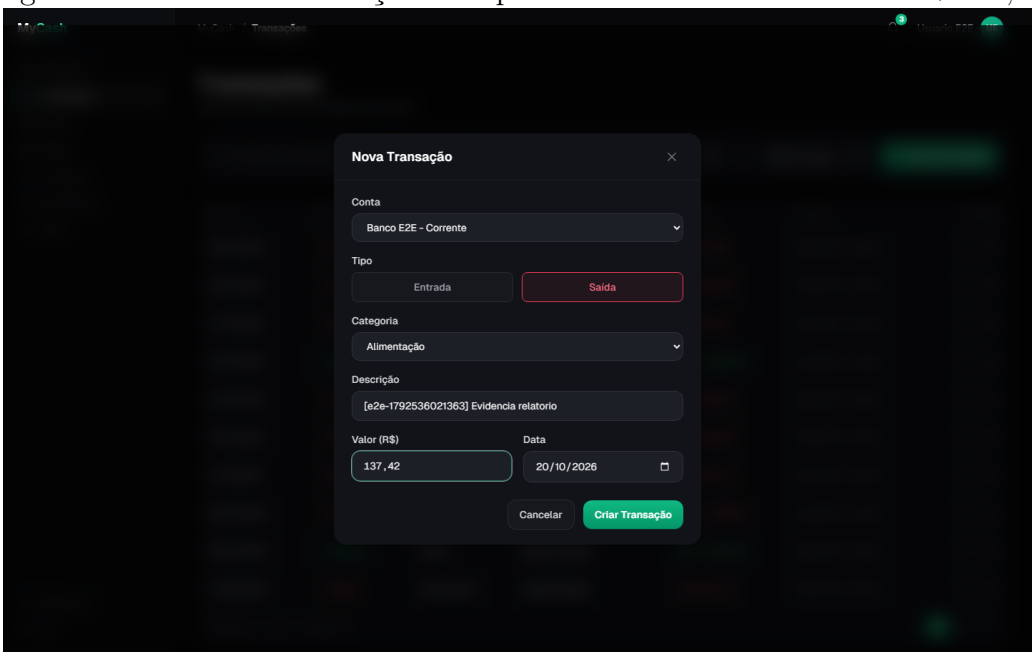



Fonte: Autores (2026).

## 4.5 Etapa 5 — Formulário de lançamento

O formulário exige conta, tipo, categoria, descrição, valor e data. As validações executadas antes do envio pertencem à camada de domínio: valor positivo obrigatório e conta em estado operável.

Figura 6: Formulário de lançamento preenchido com uma saída de R$ 137,42.

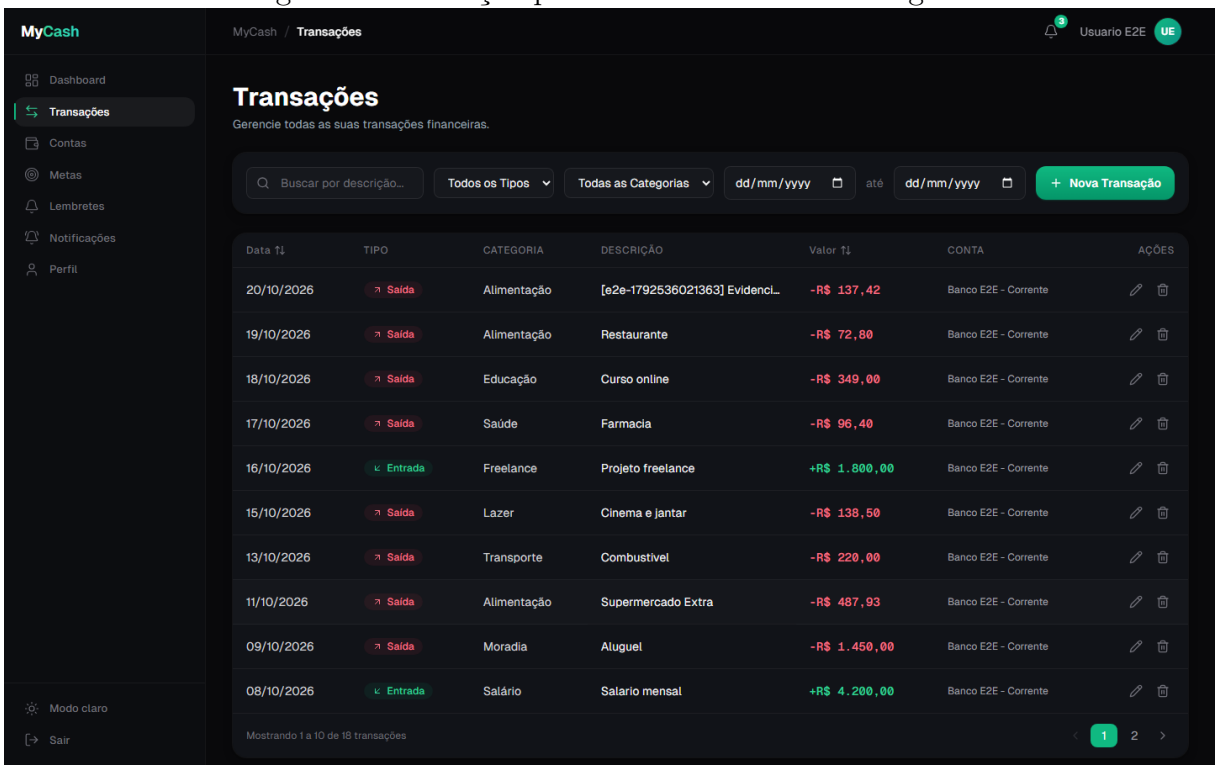



Fonte: Autores (2026).

## 4.6 Etapa 6 — Persistência

Confirmado o lançamento, a transação é persistida e passa a constar na listagem.

Figura 7: Transação persistida e exibida na listagem.

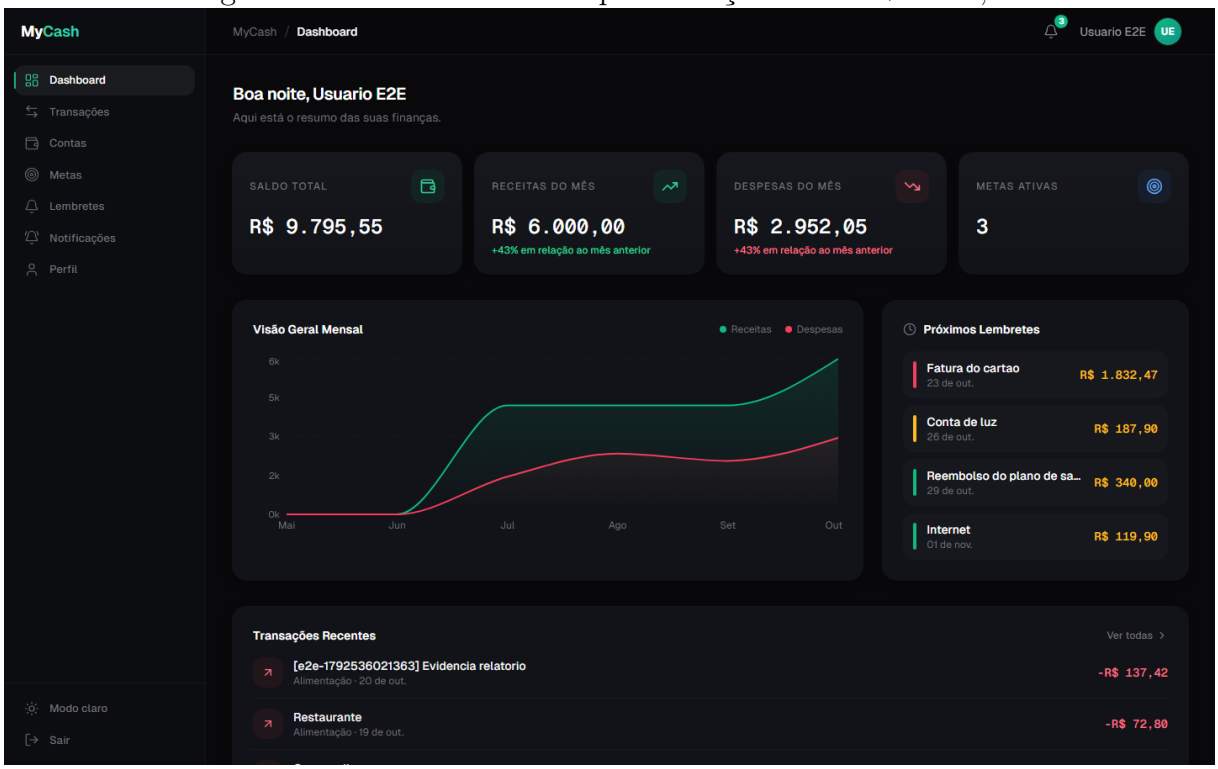



Fonte: Autores (2026).

## 4.7 Etapa 7 — Consolidação no saldo

Esta é a etapa crítica do processo. O saldo consolidado deve refletir o lançamento após recarregamento completo da página — e não apenas no estado local da interface. O saldo passou de R$ 9.932,97 para R$ 9.795,55, diferença de exatamente R$ 137,42.

Figura 8: Saldo consolidado após o lançamento: R$ 9.795,55.

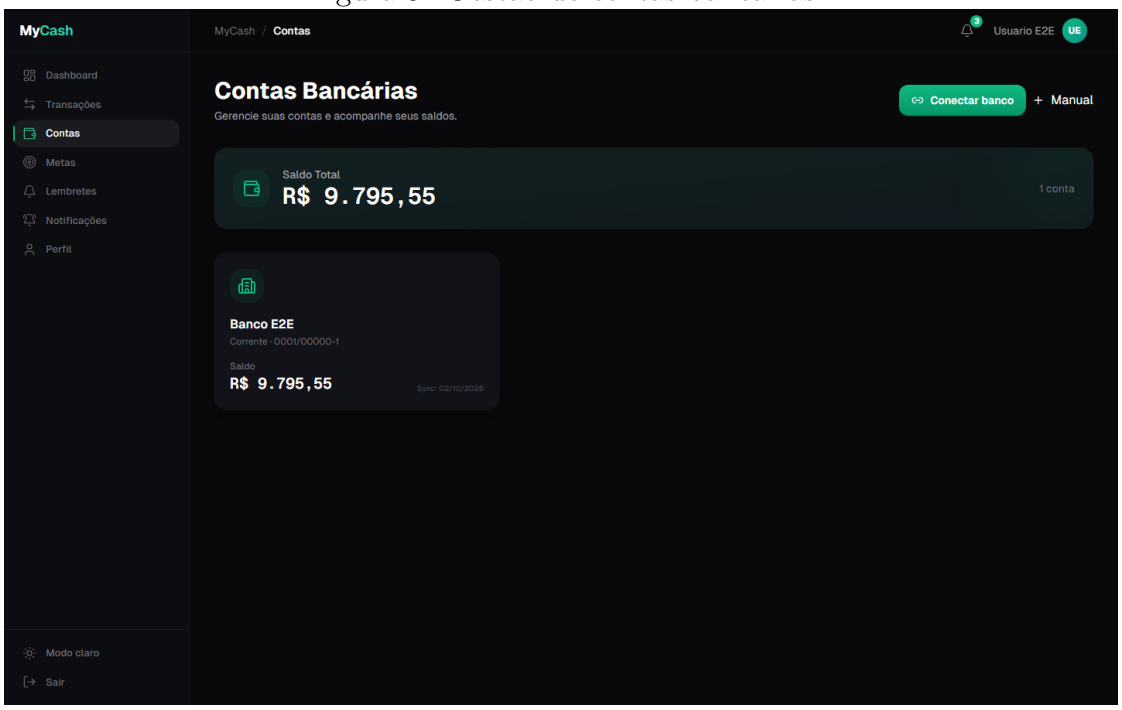



Fonte: Autores (2026).

A exigência de recarregamento não é arbitrária. Em agosto de 2026, o *dashboard* apresentou saldo congelado em produção enquanto o banco de dados já registrava o valor correto. A causa foi o *Data Cache* do Next.js, que armazenava em cache as chamadas `fetch` realizadas internamente pela biblioteca do Supabase. Como cada URL do PostgREST constitui uma chave de cache distinta, parte da tela atualizava e parte permanecia congelada. Um teste que observasse apenas o estado imediatamente posterior à mutação não detectaria o defeito.

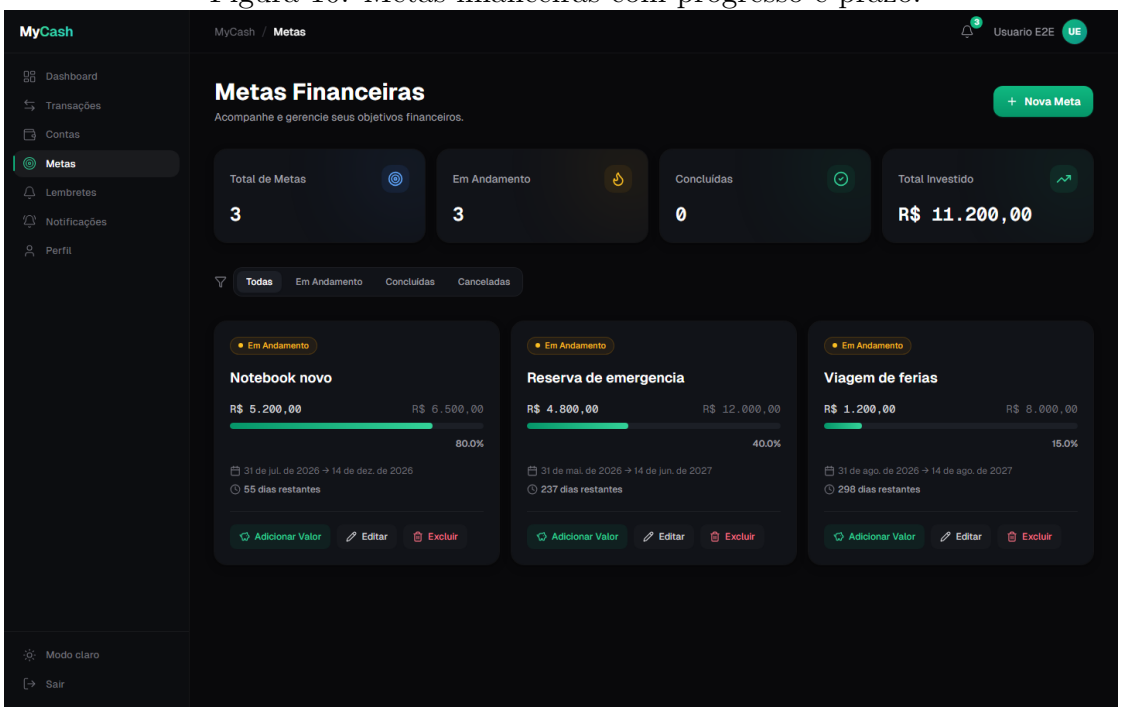
## 4.8 Demais telas do domínio

Figura 9: Gestão de contas bancárias.



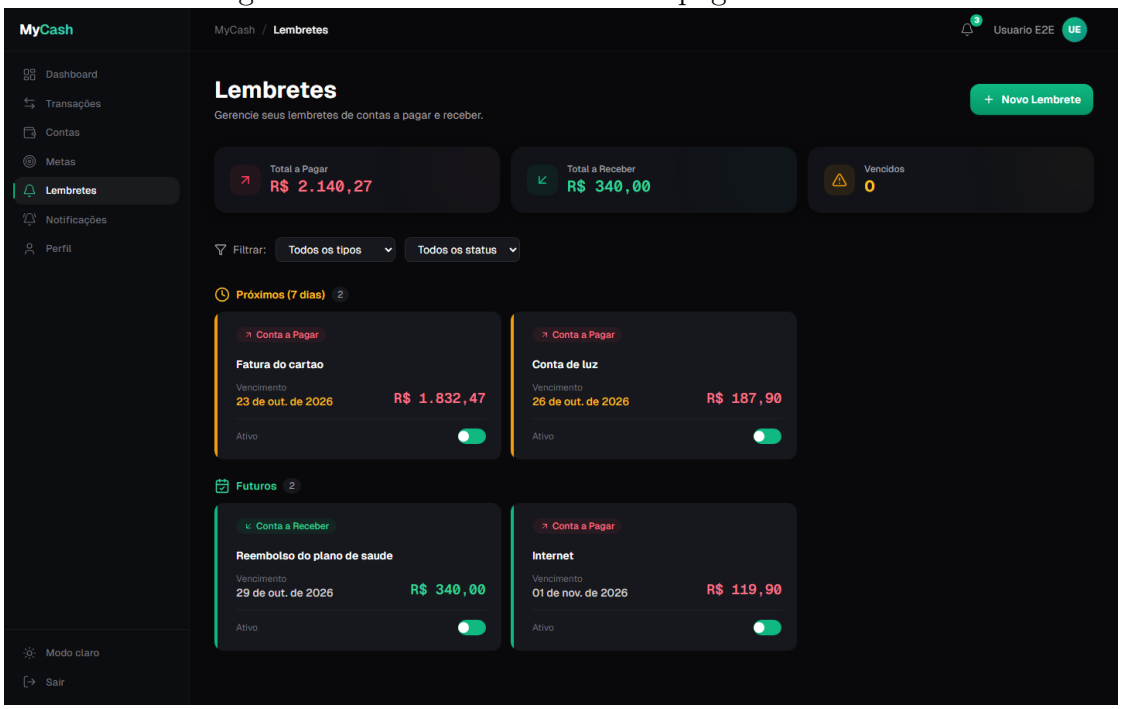

Fonte: Autores (2026).

Figura 10: Metas financeiras com progresso e prazo.



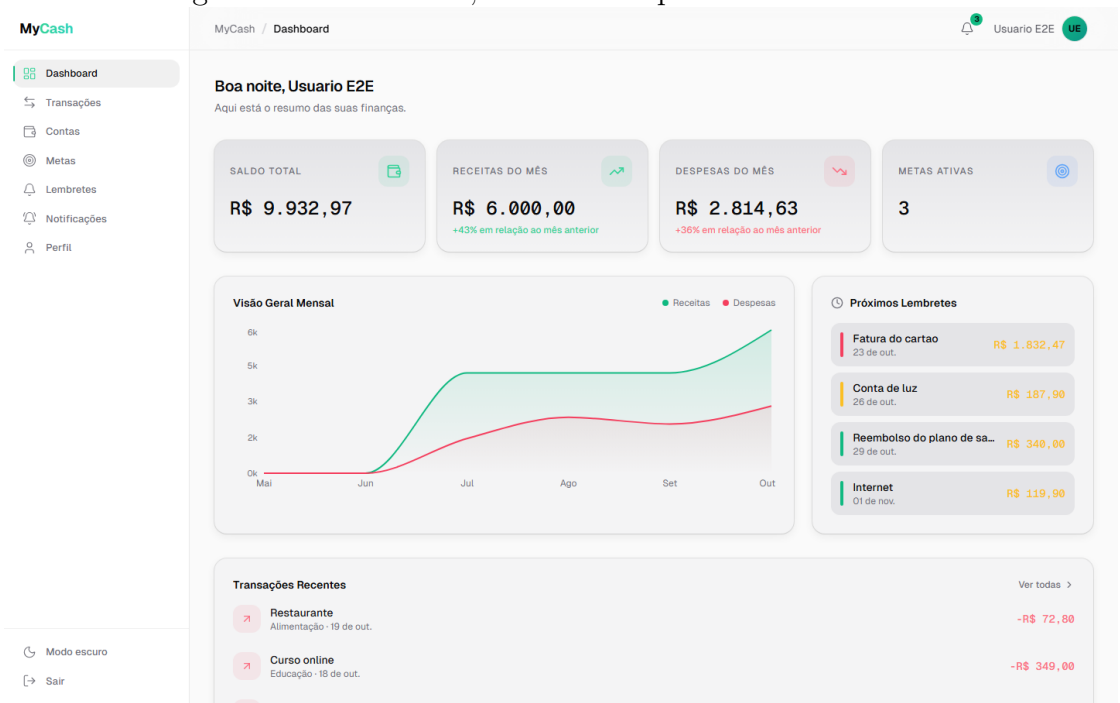

Fonte: Autores (2026).

Figura 11: Lembretes de contas a pagar e a receber.




Fonte: Autores (2026).

Figura 12: Tema claro, alternável a partir da barra lateral.




Fonte: Autores (2026).

## 4.9 Discussão

Esta seção registra as principais dificuldades enfrentadas e os defeitos revelados pelo processo de verificação. Todos os casos relatados ocorreram efetivamente durante o desenvolvimento.

**Defeito 1 — Conta bloqueada aceitava lançamentos.** A coluna `status_conta` existia no *schema* desde o início do projeto, mas nenhum ponto do sistema a consultava. Era possível lançar normalmente em conta de usuário bloqueado. O defeito só se tornou visível quando a regra foi escrita explicitamente como função de domínio e submetida a teste. Bloquear uma conta que continua recebendo movimentação não bloqueia coisa alguma.

**Defeito 2 — Transações do dia 1 contabilizadas no mês anterior.** O agrupamento mensal utilizava `new Date(string)` diretamente. Para a entrada `"2026-09-01"`, o JavaScript interpreta a data como UTC à meia-noite; no fuso de São Paulo, isso corresponde a 31 de agosto às 21 horas. Toda transação do primeiro dia do mês caía no agrupamento do mês anterior. A correção introduziu a função `paraDataLocal`, que decompõe a *string* em ano, mês e dia antes de construir a data.

**Defeito 3 — Gráfico que fabricava dados.** O gráfico de seis meses do *dashboard* web desenhava os cinco meses anteriores a partir do mês corrente, por meio da expressão `monthIncome * (0.85 + seed % 5 * 0.07)`. O resultado apresentava forma plausível e valores inteiramente inventados. Os cartões de receitas e despesas, pelo mesmo mecanismo, exibiam as *strings* fixas `"+12%"` e `-8%"` independentemente dos dados. Em um aplicativo de finanças, gráfico que inventa dado é pior do que gráfico vazio, pois parece verdadeiro. A correção consistiu em consumir a função `serieMensal` da camada de domínio — a mesma que o aplicativo móvel já utilizava — e calcular a variação real contra o mês anterior.

**Defeito 4 — Tema aplicado apenas parcialmente.** A função que montava os estilos não começava com o prefixo `use`. O React Compiler, por consequência, tratou-a como função comum e memorizou seu resultado, congelando a primeira paleta carregada. A correção foi renomeá-la para `useEstilos`, respeitando a convenção de nomenclatura de *hooks*.

**Defeito 5 — Tela branca por assimetria de versões.** Ao adicionar o campo `serieMensal` à resposta de `GET /api/dashboard`, o aplicativo móvel quebrou com `TypeError: Cannot read property 'length' of undefined`. O pacote do aplicativo já esperava o campo novo, enquanto a Vercel ainda servia a versão anterior da rota. A correção tornou o aplicativo um leitor tolerante, e o episódio originou o registro de decisão arquitetural sobre contratos multicanal.

**Defeito 6 — Rótulos sem associação acessível.** A escrita dos testes de ponta a ponta revelou que os cartões do *dashboard* eram elementos `div` anônimos e que os elemen-

tos `label` do formulário de lançamento não possuíam o atributo `htmlFor`. Visualmente pareciam rótulos, mas não o eram: um leitor de tela anunciava "caixa de edição"sem informar de qual campo. A correção acrescentou `role="group"` com `aria-label` aos cartões e associou cada rótulo ao respectivo campo.

**Entrave 1 — O *schema* nunca versionado.** As seis tabelas centrais existiam apenas no painel do Supabase, sem migração correspondente no repositório. A dívida só se tornou intransponível quando os testes de integração passaram a provisionar um PostgreSQL vazio: sem *schema* versionado, não havia o que testar. A migração foi escrita a partir dos tipos TypeScript e do uso observado nas rotas, com os pontos inferidos explicitamente marcados no arquivo.

**Entrave 2 — Diagnósticos equivocados antes da causa real.** Dois incidentes consumiram tempo considerável por hipóteses incorretas. No caso do saldo congelado, foram investigados cache de CDN e comportamento do cliente HTTP antes de se identificar o *Data Cache* do Next.js. No caso da falha da integração contínua, foram investigados propagação de variáveis de ambiente e limite de requisições do Docker Hub antes de o registro de execução revelar a causa: o ambiente de CI utilizava Node 20, enquanto a máquina de desenvolvimento utilizava Node 22, e a biblioteca `undici` do Testcontainers depende de uma API disponível apenas a partir da versão 22. A lição registrada é que o registro de execução completo deve ser consultado antes de qualquer hipótese.

**Entrave 3 — O próprio teste confirmando seu fundamento.** A primeira execução da suíte de integração falhou em 5 dos 12 casos, com violação de restrição de unicidade. A rotina de limpeza entre casos havia sido escrita, mas não conectada ao gatilho de preparação. O sintoma foi exatamente aquele que o registro de decisão arquitetural previa: teste herdando o estado deixado pelo anterior.

# 5 RELATÓRIO DE TESTES

## 5.1 Ferramentas utilizadas

As ferramentas adotadas em cada camada de verificação são apresentadas na Tabela 5.

Tabela 5: Ferramentas de verificação adotadas.

| Ferramenta | Versão | Função |
|---|---|---|
| **Vitest** | 4.1.9 | Execução das suítes de unidade e integração. |
| **@vitest/coverage-v8** | 4.1.9 | Relatório de cobertura, com limites verificados na CI. |
| **Stryker Mutator** | 10.0 | Teste de mutação sobre a camada de domínio. |
| **Testcontainers** | 12.1 | Provisionamento de PostgreSQL efêmero para a integração. |
| **Playwright** | 1.56 | Testes de ponta a ponta em navegador real. |
| **Docker Compose** | v2 | Topologia local declarativa. |
| **GitHub Actions** | — | Integração contínua com três trabalhos paralelos. |

Fonte: Autores (2026).

A estratégia segue a pirâmide de testes: muitos testes rápidos na base, poucos testes caros no topo. A suíte de unidade completa executa em aproximadamente 6 segundos sem qualquer acesso à rede; a de integração, em 7,5 segundos com um contêiner PostgreSQL; a de ponta a ponta, em 70 segundos com navegador real.

## 5.2 Testes de unidade

### 5.2.1 Quantidade e distribuição

A suíte de unidade totaliza **222 casos**, distribuídos conforme a Tabela 6.

Tabela 6: Distribuição dos testes de unidade.

| **Arquivo** | **Casos** | **Alvo** |
|---|---|---|
| `src/domain/saldo.test.ts` | 26 | Efeito no saldo, deltas, validação |
| `src/domain/metas.test.ts` | 25 | Progresso, aporte, prazo |
| `src/domain/resumo.test.ts` | 25 | Totais, série mensal, vencimentos |
| `src/domain/lancamentos.test.ts` | 21 | Caso de uso com duplos |
| `src/domain/valores.test.ts` | 18 | Conversão e arredondamento |
| `src/domain/datas.test.ts` | 15 | Fuso horário e diferenças |
| `src/lib/pluggy/mapping.test.ts` | 19 | Tradução Open Finance |
| `src/lib/api/repositories.test.ts` | 11 | Regras dos repositórios |
| `src/lib/api/client.test.ts` | 8 | Cliente HTTP e erros |
| `mobile/constants/mycash.test.ts` | 25 | Formatação e rótulos |
| `mobile/lib/repositories.test.ts` | 22 | Repositórios do aplicativo |
| `mobile/lib/api.test.ts` | 13 | Cliente HTTP do aplicativo |
| **Total** | **222** | |

Fonte: Autores (2026).

### 5.2.2 Cobertura

O resultado consolidado consta da Tabela 7. A cobertura é medida exclusivamente sobre `src/domain`, a camada que a suíte de unidade tem como alvo. Medir `src/` integralmente diluiria o denominador com páginas e componentes do Next, que teste de unidade não exercita, produzindo um número baixo e sem significado que esconderia lacunas reais no domínio.

Tabela 7: Relatório de cobertura da camada de domínio.

| **Métrica** | **Cobertura** | **Absoluto** |
|---|---|---|
| Instruções (*statements*) | 100% | 112 / 112 |
| Ramos (*branches*) | 100% | 73 / 73 |
| Funções | 100% | 38 / 38 |
| Linhas | 100% | 100 / 100 |

Fonte: Autores (2026).

Os limites de cobertura estão declarados em `vitest.config.ts` e verificados a cada envio: um resultado abaixo do limite reprova o trabalho de integração contínua. Trata-se de uma trava, não de uma meta — a distinção é relevante e retomada na Seção 5.2.4.

O relatório cumpriu sua função concreta. A primeira execução apontou 91,78% de cobertura de ramos, identificando seis ramos jamais executados — todos guardas contra valor nulo proveniente do PostgREST. Seis casos adicionais foram escritos, fechando os seis ramos.

### 5.2.3 Cenários alternativos

A suíte não se limita ao caminho feliz. Dos 222 casos, aproximadamente 40% exercitam condições de erro, valores de fronteira ou entradas malformadas. A Tabela 8 apresenta exemplos representativos.

Tabela 8: Exemplos de cenários alternativos cobertos.

| **Cenário** | **Comportamento esperado** |
|---|---|
| Valor zero ou negativo | Lança `ValorInvalidoError` antes de qualquer persistência. |
| Conta bloqueada ou inativa | Lança `ContaBloqueadaError`, com mensagem nomeando a instituição. |
| Entrada `"12abc"` | Resulta em `NaN`, não em 12. |
| Transação no dia 1 do mês | Agrupada no mês correto, apesar do fuso horário. |
| Coluna nula vinda do banco | Tratada como zero, sem quebrar a soma. |
| Aporte acima do objetivo | Limitado ao objetivo; meta marcada como concluída. |
| Prazo de meta invertido | Recusado com `DataInvalidaError`. |
| Exclusão repetida | Não debita o valor duas vezes. |
| *Subtype* desconhecido da Pluggy | Convertido para `"Corrente"`, nunca `undefined`. |

Fonte: Autores (2026).

### 5.2.4 Teste de mutação

Cobertura de 100% indica que todas as linhas foram executadas, mas não que alguma asserção verificou o resultado. Um teste que invoque a função e nada afirme produz cobertura idêntica à de um teste rigoroso. É a Lei de Goodhart aplicada à métrica: quando a cobertura vira meta, ela deixa de ser boa medida.

O teste de mutação responde a essa limitação. O Stryker introduz alterações deliberadas no código — troca `>` por `>=`, inverte condições, substitui valores por constantes — e verifica se algum teste falha. Mutante que sobrevive indica comportamento não verificado. O resultado por arquivo consta da Tabela 9.

Tabela 9: Escore de mutação por arquivo do domínio.

| Arquivo | Escore | Mortos | Sobreviventes |
|---|---|---|---|
| `saldo.ts` | 96,15% | 50 | 2 |
| `valores.ts` | 96,43% | 27 | 1 |
| `datas.ts` | 94,74% | 18 | 1 |
| `resumo.ts` | 93,33% | 70 | 5 |
| `lancamentos.ts` | 93,10% | 27 | 2 |
| `metas.ts` | 92,00% | 46 | 4 |
| **Total** | **94,07%** | **238** | **15** |

Fonte: Autores (2026).

Dos 253 mutantes gerados, 238 foram eliminados e 15 sobreviveram, sem erros nem esgotamentos de tempo. O escore partiu de 89,84%, atingiu 92,09% após a primeira rodada de correções e alcançou 94,07% após a inclusão dos testes de guarda contra valor nulo.

A análise dos sobreviventes também produziu conhecimento. Um deles apontava uma verificação explícita de `NaN` na função de série mensal: o mutante sobrevivia porque a verificação era código inalcançável — data inválida já produzia chave inexistente no mapa, e o próprio mapa descartava o lançamento. O código redundante foi removido.

## 5.3 Testes de integração

A suíte de integração totaliza **35 casos**, executados contra um PostgreSQL 17 real provisionado por Testcontainers. O contêiner sobe uma vez por execução, no `globalSetup`, e é descartado ao final.

### 5.3.1 Estratégia de isolamento de estado

A estratégia adotada combina **banco por trabalhador** e **truncamento entre casos**, decisão registrada em documento de decisão arquitetural. As alternativas consideradas constam da Tabela 10.

Tabela 10: Estratégias de isolamento consideradas.

| Estratégia | Avaliação |
|---|---|
| **Reversão por transação** | Mais barata, porém exigiria que o código de produção recebesse a conexão da transação. As rotas instanciam o próprio cliente Supabase; adaptar significaria alterar toda a camada de acesso para servir ao teste. Descartada por acoplamento, não por desempenho. |
| **Truncamento por teste** | Resolve a ordem de execução, mas não o paralelismo. |
| **Banco por trabalhador** | Resolve o paralelismo, mas não isola casos dentro do mesmo trabalhador. |
| **Adotada: as duas últimas combinadas** | Um contêiner, um banco por trabalhador do Vitest criado a partir de um modelo com o *schema* já aplicado, e truncamento com reinício de sequências entre casos. |

Fonte: Autores (2026).

### 5.3.2 Escopo coberto

**Banco de dados (12 casos).** Validam a própria migração: criação das seis tabelas centrais e dos seis tipos enumerados, ordem correta de aplicação das três migrações, restrições `CHECK`, chaves estrangeiras, unicidade de *e-mail*, cascatas de remoção e ausência de resíduo binário em colunas `numeric(14,2)`.

**API própria e caso de uso (12 casos).** Exercitam o caso de uso de lançamento com adaptadores PostgreSQL reais, e não com duplos. É a fronteira que a unidade não alcança: o controlador `pg` devolve colunas `numeric` como *string*, e não como número — comportamento que duplo algum revelaria.

**API externa, Open Finance (11 casos).** Verificam a idempotência da sincronização. O processo reprocessa as mesmas transações a cada execução — webhook reentregue, acionamento manual, página que retorna integralmente. O que impede a duplicação não é código TypeScript, e sim a restrição `UNIQUE` sobre `pluggy_tx_id` combinada com `ON CONFLICT DO UPDATE`.

Listing 2: Verificação de idempotência contra o banco real.

```
it('sync_mesmaTransacaoDuasVezes_naoDuplicaALinha', async () => {
  await sincronizarTransacao({ pluggyTxId: 'tx-abc', valor: 150 })
  await sincronizarTransacao({ pluggyTxId: 'tx-abc', valor: 150 })

  expect(await contarTransacoes()).toBe(1)
```

```
})
```

Esta suíte revelou uma propriedade que somente um banco real evidencia: a restrição `UNIQUE` do PostgreSQL admite múltiplos valores nulos. É precisamente isso que permite a convivência, na mesma tabela, entre lançamentos manuais, cujo `pluggy_tx_id` é nulo, e transações sincronizadas. Sem essa característica, a coluna limitaria o sistema a um único lançamento manual.

**Observação quanto ao escopo.** O que não está verificado é a forma da resposta real da Pluggy. Os testes exercitam a tradução e a persistência próprias; nada garante que o provedor mantenha o formato que a camada de tradução espera. Trata-se do caso clássico de contrato dirigido pelo consumidor, e é a lacuna conhecida registrada na Tabela 3.

## 5.4 Testes de ponta a ponta

A suíte de ponta a ponta totaliza **9 casos** em Playwright, organizados segundo o padrão Page Object Model. Os localizadores utilizam papel semântico ARIA e texto visível, nunca classes CSS ou expressões XPath — as classes são utilitárias do Tailwind, mudam ao primeiro ajuste visual e não descrevem função alguma.

Os testes executam contra a imagem de produção servida pelo contêiner Docker, e não contra o servidor de desenvolvimento. Os dados de partida são criados por chamada à API administrativa do Supabase, prática denominada *API Data Seeding*: o estado inicial provém de chamada determinística, e não de jornada pela interface nem de registro fixo que alguém possa ter removido.

### 5.4.1 Cenário crítico 1 — Portal de autenticação

Enquadra-se no critério de portais de autenticação e integração: mecanismo de entrada cuja indisponibilidade bloqueia integralmente o acesso. No MyCash não há caminho alternativo — toda rota de dados está atrás da sessão, e a *Row Level Security* filtra por usuário. Os seis casos que compõem o cenário constam da Tabela 11.

Tabela 11: Cenário crítico 1 — casos e respectivos caminhos.

| **Caso** | **Caminho** | **Verificação** |
|---|---|---|
| Formulário acessível | Feliz | Campos localizáveis por rótulo e papel. |
| Senha não visível na tela | Feliz | Atributo `type="password"` preservado. |
| *E-mail* malformado | Alternativo | Bloqueado pela validação do navegador. |
| Credencial inválida | Alternativo | Erro exibido; permanece em `/login`. |
| Rota protegida sem sessão | Alternativo | Redirecionado para `/login`. |
| Recuperação de senha | Feliz | Conduz ao fluxo de recuperação. |

Fonte: Autores (2026).

Destaca-se o caso de rota protegida sem sessão: é ele que efetivamente fecha o portão. Caso o *middleware* falhe, a tela passaria a ser renderizada e o vazamento apareceria aqui — nenhuma camada inferior enxerga *middleware*.

### 5.4.2 Cenário crítico 2 — Caminho transacional

Ocupa o lugar do caminho crítico de receita. O MyCash não gera receita, mas o equivalente funcional é direto: um aplicativo de finanças cujo saldo não corresponde à realidade não cumpre função alguma. Os três casos constam da Tabela 12.

Tabela 12: Cenário crítico 2 — casos e respectivos caminhos.

| **Caso** | **Caminho** | **Verificação** |
|---|---|---|
| Saída reduz o saldo consolidado | Feliz | Saldo e despesas do mês após recarregamento. |
| Entrada aumenta o saldo consolidado | Feliz | Saldo e receitas do mês após recarregamento. |
| Criação seguida de exclusão | Alternativo | Saldo retorna ao valor inicial. |

Fonte: Autores (2026).

A escolha deste cenário não é teórica: o fluxo apresentou falha em produção por duas vezes, descritas na Seção 4.9 como Defeitos 3 e 5, e em ambas as ocasiões a totalidade da suíte permanecia aprovada. O domínio estava correto, com 222 testes de unidade

passando; o PostgreSQL estava correto, com 35 testes de integração passando. O defeito residia entre o navegador e a API — faixa que somente esta camada alcança.

### 5.4.3 Artefatos de diagnóstico

Em caso de falha, o Playwright retém *trace* compactado, vídeo da execução e captura de tela. O *trace* preserva DOM, tráfego de rede, console e captura por passo, e é inspecionado por meio do comando `npx playwright show-trace`. A retenção ocorre apenas em falha: gravar sempre consumiria disco sem contrapartida.

## 5.5 Deliberações sobre o que não entra em cada camada

A pirâmide impõe disciplina quanto ao que *não* pertence a cada nível. A Tabela 13 registra as decisões tomadas.

Tabela 13: Verificações deliberadamente mantidas fora da camada de ponta a ponta.

| **Verificação** | **Camada responsável** |
|---|---|
| Valor zero ou negativo recusado | Unidade (`ValorInvalidoError`) |
| Conta bloqueada recusa lançamento | Unidade e integração |
| Arredondamento em centavos | Integração (`numeric(14,2)`) |
| Idempotência da sincronização | Integração (`UNIQUE` e `ON CONFLICT`) |
| Cascatas de remoção | Integração |

Fonte: Autores (2026).

Cada um desses casos, se replicado na camada de ponta a ponta, custaria segundos de navegador para comprovar o que já está comprovado em milissegundos.

# 6 ESTRATÉGIA DE IMPLEMENTAÇÃO

## 6.1 Nuvem ou *on-premises*

A implantação é integralmente **em nuvem**, segundo o modelo de plataforma como serviço. Não há servidor físico ou virtual sob administração da equipe. A Tabela 14 descreve a distribuição dos componentes.

Tabela 14: Componentes de implantação e respectivos provedores.

| Componente | Provedor | Justificativa |
|---|---|---|
| **Aplicação web** | Vercel | Implantação contínua a partir do ramo principal; funções sem estado; ambiente de pré-visualização por *pull request*. |
| **Banco de dados** | Supabase (AWS, `sa-east-1`) | PostgreSQL gerenciado com *Row Level Security*, API REST automática e autenticação integrada. |
| **Autenticação** | Supabase Auth | Confirmação por *e-mail*, recuperação de senha e renovação de sessão sem código próprio. |
| **Open Finance** | Pluggy | Agregador homologado junto ao Banco Central. |
| **Aplicativo móvel** | Expo | Distribuição por *bundle* e atualização sem nova submissão às lojas. |

Fonte: Autores (2026).

**Justificativa da escolha.** Para equipe de quatro estudantes em projeto acadêmico, administrar servidor próprio consumiria tempo de desenvolvimento sem agregar aprendizado proporcional. A dependência de provedor é real e foi assumida conscientemente; a contenção adotada é que a lógica de negócio reside em uma camada que nada conhece sobre Supabase, o que torna uma eventual troca de provedor uma alteração de adaptador, e não uma reescrita.

## 6.2 Infraestrutura como código

Ainda que a produção seja gerenciada, o ambiente local foi declarado como código. A motivação é a eliminação da deriva de configuração: ambiente montado manualmente diverge progressivamente da especificação, e cada máquina de desenvolvimento torna-se um caso particular.

### 6.2.1 Imagem de produção

O `Dockerfile` emprega três estágios. Compilar e executar possuem necessidades opostas: compilar requer cadeia de ferramentas C/C++ e a árvore completa de dependências; executar requer apenas o ambiente de execução Node e o servidor gerado. Reunir ambos em uma única imagem significaria embarcar compilador em produção. A Tabela 15 descreve a finalidade de cada estágio.

Tabela 15: Estágios da imagem de produção.

| Estágio | Conteúdo e finalidade |
|---|---|
| **deps** | Instala `python3`, `make` e `g++` para dependências com extensão nativa, e executa `npm ci`. Recebe apenas os manifestos, de modo que a camada seja reaproveitada enquanto as dependências não mudarem. |
| **builder** | Recebe as dependências prontas e executa `npm run build`, gerando a saída *standalone*. |
| **runner** | Não herda compilador nem árvore de desenvolvimento. Recebe somente `.next/standalone` e `.next/static`, e executa sob usuário `nextjs` de identificador 1001. |

Fonte: Autores (2026).

A imagem final resultante ocupa **356 MB**. A execução sob usuário sem privilégios é medida de contenção: o Docker executa como `root` por padrão, e uma falha de execução remota de código na aplicação se converteria em `root` dentro do contêiner.

Listing 3: Trecho do estágio final da imagem.

```
FROM node:22.14.0-bookworm-slim AS runner
WORKDIR /app

ENV NODE_ENV=production
ENV HOSTNAME="0.0.0.0"

RUN addgroup --system --gid 1001 nodejs \
    && adduser --system --uid 1001 --ingroup nodejs nextjs

COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static
   ./.next/static

USER nextjs
EXPOSE 3000
CMD ["node", "server.js"]
```

A versão da imagem base está fixada com número de correção — `22.14.0`, e não `22` — porque rótulo móvel reintroduz exatamente a deriva que a infraestrutura imutável existe para eliminar: a mesma linha produziria imagens distintas em datas distintas.

### 6.2.2 Topologia local

O manifesto `compose.yaml` declara os três serviços apresentados na Tabela 16, operáveis por meio de um único comando: `docker compose up -build`.

Tabela 16: Serviços declarados no manifesto Compose.

| Serviço | Função |
|---|---|
| `web` | Aplicação Next.js em modo *standalone*, sob usuário sem privilégios, com sonda de integridade em `/api/health`. |
| `db` | PostgreSQL 17 com volume nomeado `mycash-db-data` e sonda de integridade via `pg_isready`. |
| `migrate` | Contêiner de tarefa que aplica as três migrações na ordem correta e encerra. Depende de `db` com condição `service_healthy`. |

Fonte: Autores (2026).

**Observação sobre a ausência de um serviço de API separado.** No MyCash, a API corresponde às rotas do próprio Next, executadas no mesmo processo do servidor. Não existe um segundo ambiente de execução Node a ser empacotado. Declarar um contêiner adicional apenas para compor um diagrama com duas caixas resultaria em serviço fictício, e a decisão foi registrada em vez de dissimulada.

**Sobre o serviço de banco de dados.** O PostgreSQL local não constitui o *backend* da aplicação — em produção, o MyCash utiliza Supabase gerenciado, e a biblioteca cliente comunica-se por PostgREST, não por SQL direto. O papel deste serviço é outro, e igualmente concreto: constitui o ambiente onde o *schema* versionado é aplicado e pode ser inspecionado sem qualquer contato com produção. É precisamente o que faltava quando as seis tabelas centrais existiam somente no painel do provedor.

**Persistência do volume.** O uso de volume nomeado, em lugar de montagem de diretório do sistema hospedeiro, é o que assegura a sobrevivência dos dados ao comando `docker compose down`. A verificação foi realizada: um registro gravado permaneceu disponível após a destruição e recriação dos contêineres.

### 6.2.3 Configuração e segredos

O arquivo `.env.example` documenta a distinção mais relevante da configuração no Next.js, apresentada na Tabela 17.

Tabela 17: Variáveis de compilação e variáveis de execução.

| Prefixo | Semântica |
|---|---|
| `NEXT_PUBLIC_` | Variável de **compilação**. O compilador substitui cada ocorrência pelo valor literal no interior do pacote JavaScript. Após a compilação, o valor encontra-se no arquivo que o navegador transfere, acessível a qualquer pessoa. |
| `sem prefixo` | Variável de **execução**. Permanece restrita ao processo Node do servidor e jamais é enviada ao navegador. |

Fonte: Autores (2026).

Decorre daí a regra adotada: nenhum segredo recebe o prefixo `NEXT_PUBLIC_`. A chave `service_role` do Supabase ignora a *Row Level Security*; prefixá-la equivaleria a publicar na internet acesso irrestrito ao banco de todos os usuários. A *anon key*, em contraste, é pública por concepção — quem protege os dados são as *policies*, não o sigilo da chave.

## 6.3 Integração contínua com GitHub Actions

O fluxo de trabalho `.github/workflows/ci.yml` é acionado a cada envio e a cada *pull request* dirigidos ao ramo principal, executando os três trabalhos em paralelo descritos na Tabela 18.

Tabela 18: Trabalhos da integração contínua.

| Trabalho | Etapas |
|---|---|
| **web** | Verificação de tipos (`tsc -noEmit`), análise estática (`next lint`), testes com cobertura (`npm run test:cov`) e publicação do relatório de cobertura como artefato. |
| **mobile** | Verificação de tipos, análise estática e testes do aplicativo, com árvore de dependências própria. |
| **integracao** | Transferência prévia da imagem `postgres:17-alpine` e execução da suíte de integração com Testcontainers. |

Fonte: Autores (2026).

**Decisões de configuração registradas.** A separação entre web e móvel decorre de árvores de dependências incompatíveis. A separação do trabalho de integração decorre

do custo: ele provisiona um contêiner PostgreSQL, e não faria sentido reter o retorno da análise estática à espera dessa execução. A transferência da imagem ocorre em etapa própria para que eventual falha de obtenção se manifeste explicitamente, em lugar de converter-se em esgotamento de tempo na suíte. O contêiner auxiliar de limpeza do Testcontainers é desabilitado no ambiente de integração, onde a máquina é descartada integralmente ao término. O paralelismo é reduzido a dois trabalhadores, correspondendo ao número de núcleos do executor.

**Correção registrada.** Os três trabalhos utilizam Node 22. Com Node 20, o trabalho de integração falhava com `TypeError: webidl.util.markAsUncloneable is not a function`: a biblioteca `undici`, dependência do Testcontainers, emprega uma API disponível somente a partir da versão 22. O sintoma era enganoso — o Vitest encerrava durante o carregamento da configuração global e reportava "No test files found", como se o problema residisse no padrão de inclusão de arquivos.

**Rotina complementar.** Um segundo fluxo de trabalho, `supabase-keepalive.yml`, executa diariamente uma consulta real ao PostgREST, impedindo a suspensão automática do projeto por inatividade, característica do plano gratuito da plataforma.

## 6.4 Entrega contínua

A Vercel observa o ramo principal e implanta automaticamente cada alteração aprovada pela integração contínua. Cada *pull request* recebe ambiente de pré-visualização com URL própria. A configuração `output: 'standalone'` é ignorada pela Vercel, que realiza o próprio empacotamento, de modo que a imagem Docker e a implantação em produção coexistem sem conflito.

# 7 CONSIDERAÇÕES PARCIAIS

A etapa documentada neste relatório alterou a natureza do projeto. O módulo anterior entregou uma aplicação que funcionava; esta entregou os meios de demonstrar que ela funciona, e de reproduzir o ambiente em que funciona.

O resultado mais consistente não está nos números, e sim no que o processo revelou. Seis defeitos reais foram identificados em código que se encontrava em produção e aparentava correção — entre eles contas bloqueadas que aceitavam lançamentos, transações do primeiro dia do mês contabilizadas no mês anterior e um gráfico que fabricava os meses anteriores. Nenhum deles havia sido percebido em uso normal, e nenhum teria sido encontrado sem a disciplina de escrever a verificação antes de confiar no comportamento observado.

Três aprendizados merecem registro. Primeiro, a extração da camada de domínio não foi motivada por elegância arquitetural, e sim pela constatação concreta de que a mesma

regra, escrita duas vezes, havia divergido em quatro pontos. Segundo, cobertura de 100% não significa código verificado: foi o teste de mutação que revelou asserções ausentes e código inalcançável. Terceiro, cada camada da pirâmide encontra defeitos que as demais não alcançam, e os dois incidentes de produção só seriam detectáveis pela camada de ponta a ponta, construída por último.

## 7.1 Próximos passos

- Formalizar contratos dirigidos pelo consumidor, com três participantes distintos: aplicação web, aplicativo Android e aplicativo iOS;
- Validar a integração Open Finance contra a conta *sandbox* da Pluggy, cobrindo a lacuna remanescente de contrato com o provedor;
- Automatizar a execução da suíte de ponta a ponta na integração contínua, com particionamento caso o tempo total ultrapasse o orçamento estabelecido;
- Incorporar observabilidade em produção para captura de erros em ambiente real;
- Concluir a migração dos ativos de hospedagem para contas sob gestão da equipe atual.

O código-fonte encontra-se publicamente disponível em `https://github.com/bernardoguerraa/mycash` e o produto em produção pode ser acessado em `https://mycash-nu.vercel.app`.

# REFERÊNCIAS


[1] ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

[2] ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10719**: informação e documentação: relatório técnico e/ou científico: apresentação. Rio de Janeiro: ABNT, 2015.

[3] BECK, K. **Test Driven Development: By Example**. Boston: Addison-Wesley, 2002.

[4] FOWLER, M. **Immutable Server**. Bliki, 2013. Disponível em: `https://martinfowler.com/bliki/ImmutableServer.html`. Acesso em: outubro 2026.

[5] FOWLER, M. **Snowflake Server**. Bliki, 2012. Disponível em: `https://martinfowler.com/bliki/SnowflakeServer.html`. Acesso em: outubro 2026.

[6] KHORIKOV, V. **Unit Testing: Principles, Practices, and Patterns**. Shelter Island: Manning, 2020.

[7] MESZAROS, G. **xUnit Test Patterns: Refactoring Test Code**. Boston: Addison-Wesley, 2007.

[8] MORRIS, K. **Infrastructure as Code**: Dynamic Systems for the Cloud Age. 2. ed. Sebastopol: O'Reilly Media, 2020.

[9] MICROSOFT. **Playwright Documentation**. Disponível em: `https://playwright.dev/docs/intro`. Acesso em: outubro 2026.

[10] STRYKER MUTATOR. **Mutation Testing Documentation**. Disponível em: `https://stryker-mutator.io/docs/`. Acesso em: outubro 2026.

[11] TESTCONTAINERS. **Testcontainers for Node.js**. Disponível em: `https://node.testcontainers.org/`. Acesso em: outubro 2026.

[12] WIGGINS, A. **The Twelve-Factor App**. 2011. Disponível em: `https://12factor.net/`. Acesso em: outubro 2026.

[13] WINTERS, T.; MANSHRECK, T.; WRIGHT, H. **Software Engineering at Google**: Lessons Learned from Programming Over Time. Sebastopol: O'Reilly Media, 2020.